

nhis-scrapper 보안 점검 결과서

문서 버전: v2 · 최종 수정일: 2026-07-08

수정 이력

버전	일자	주요 변경
v1	—	최초 작성
v2	2026-07-08	- §3 로그 마스킹에 라우트별 @UnmaskLogPaths 예외 반영

항목	내용
제품명	nhis-scrapper
버전	0.1.0
공급자	렉스소프트
점검 형태	공급자 자체 점검 (self-audit)
점검 항목	취약점 점검 · 로그 마스킹 · 데이터 암호화 · 접근 통제
함께 보아야 할 문서	../구조/아키텍처.md §5 보안 구간, ../오픈소스-라이선스-및-취약점/오픈소스-라이선스-및-취약점.xlsx

본 문서는 공급자가 본 제품의 보안 통제 구현 사실을 자체 점검한 결과를 정리한 것으로, 고객사 보안 기준이 별도로 부여되지 않은 경우의 자기증명 자료입니다.

각 항목은 「점검 방법 → 결과 → 증빙」 의 3단 구조로 기술합니다.

1. 점검 범위

본 점검은 본 제품의 컨테이너 이미지(`nhis-scrapper` , `nginx`) 와 그 안의 코드·설정, 그리고 본 제품이 강제하는 보안 통제(API key 인증, 세션 암호화, 로그 마스킹 등)에 한정합니다.

다음 항목은 본 제품의 코드·이미지에 포함되지 않는 외부 인프라 통제이므로 본 점검의 대상이 아닙니다:

- ALB · 보안그룹 · VPC 라우팅 등 네트워크 통제
- TLS 종단 위치 (외부 ↔ ALB / nginx, WAS ↔ ElastiCache 의 `REDIS_TLS`)
- ElastiCache 의 인증·암호화 활성화 여부

2. 취약점 점검 (OSS · 컨테이너 이미지)

2.1 점검 방법

- **SBOM 생성**: Docker Scout (v1.20.4) 로 두 컨테이너 이미지(`nhis-scrapper:0.1.0` , `nginx:1.27-alpine`)의 SPDX 2.3 JSON SBOM 생성
- **CVE 스캔**: Anchore Grype (v0.112.0) 로 SBOM 입력 스캔. DB schema v6.1.4 (build 2026-05-27)
- **스코프**: OS 패키지(apk) + npm 패키지 전체

2.2 결과

심각도	발견 건수
Critical	0
High	0
Medium	0
Low	0
합계	0

스캔 시점 기준 알려진 취약점 0건.

2.3 증빙

- 원본 SBOM: docs/오픈소스-라이선스-및-취약점/sbom/{nhis-scrapper,nginx}.spdx.json
- 인벤토리·취약점 표: docs/오픈소스-라이선스-및-취약점/오픈소스-라이선스-및-취약점.xlsx 시트 ③
- 라이선스 고지문: docs/오픈소스-라이선스-및-취약점/OSS-LICENSE-NOTICE.md

2.4 운영 측 권고

- 분기 단위로 동일 도구 체인으로 재스캔 (tmp/bin/grype sbom:... -o json)
- NHIS 변경·base image 패치 등으로 새 번들 수령 시 재스캔하여 산출물 갱신

3. 로그 마스킹

3.1 점검 방법

- 로그 마스킹 구현 코드 검토 (src/common/logger/)
- 인입 HTTP 요청 / WAS 내부 로깅 / NHIS 호출 시 query string 마스킹 처리 경로 추적
- 단위 테스트 (*.spec.ts) 로 마스킹 동작 검증

3.2 결과

화이트리스트(allowlist) 방식으로 비식별 키만 명시적으로 노출하고, 그 외 모든 leaf 값은 maskValue 함수로 마스킹 된다.

통제	동작
로그 메타데이터 마스킹	LOG_SAFE_KEYS 화이트리스트(message / event / step / phase 등 비식별 필드) 외 모든 leaf 값을 maskValue 가 마스킹

통제	동작
마스킹 알고리즘	첫·끝 글자만 남기고 중간을 글자수만큼 * 로 치환. 예: 홍길동 → 홍*동 , 010-1234-5678 → 0*****8
인입 요청 본문	INBOUND_LOG_ALLOWED_KEYS 별도 화이트리스트로 통제
@UnmaskLogPaths (라우트별 예외)	특정 라우트의 비민감 경로만 마스킹 예외로 원문 노출 — 예: start_auth 의 body.scope , after_auth 의 body.options . 등록되지 않은 경로·필드 및 PII 는 계속 마스킹
URL query string	maskUrlQuery() 가 모든 query 파라미터 값 마스킹 (NHIS 호출 로깅 시 적용)
응답 본문	로그 미기록 (NHIS 응답 본문은 로그에 남기지 않음. 처리되지 않은 NHIS 측 오류 응답은 NhisUpstreamHttpFailedError 의 data 필드로 전달)

3.3 증빙

- 메타데이터 화이트리스트 (LOG_SAFE_KEYS): src/common/logger/log-safe-keys.constants.ts
- 인입 요청 화이트리스트 (INBOUND_LOG_ALLOWED_KEYS): src/common/logger/log-redaction.constants.ts
- 마스킹 함수: src/common/utils/mask-value.ts , src/common/utils/redact-by-allowlist.ts
- 적용 진입점: src/common/logger/winston.logger.ts (winston format chain),
src/common/middleware/http-request-logger.middleware.ts (인입 요청 로깅)
- 라우트별 예외 (@UnmaskLogPaths): src/common/logger/unmask-log-paths.decorator.ts ,
src/common/logger/request-log-unmask.registry.ts , src/common/utils/apply-unmask-paths.ts ,
src/common/logger/safe-log-value.ts
- 검증: src/common/utils/mask-value.spec.ts , src/common/logger/winston.logger.spec.ts ,
src/common/middleware/http-request-logger.middleware.spec.ts 등 단위 테스트 통과

3.4 잔여 위험

- 표준 로거(@nestjs/common 의 Logger) 를 거치지 않고 console.log 등을 직접 호출하면 마스킹이 적용되지 않음. 코드 리뷰·정적 분석에서 통제.

4. 데이터 암호화

4.1 점검 방법

- 세션(개인정보) 의 저장 형태 추적 (ElastiCache 저장 직전·직후)
- 암호화 알고리즘·IV·인증 태그 구현 검토 (src/common/crypto/session-cipher.service.ts)
- 키 파생·관리 정책 점검

4.2 결과

세션은 메모리에서만 평문이고, ElastiCache 에는 AES-256-GCM 으로 암호화된 base64 토큰만 저장됨.

항목	값
알고리즘	AES-256-GCM (인증 암호화, 위·변조 탐지 포함)
키 길이	256 bit (32 byte)
키 파생	<code>NHIS_SESSION_ENCRYPTION_KEY</code> (32자 이상 passphrase) → <code>scriptSync(key, "nhis-scrapper-session-v1", 32 byte)</code>
Nonce (IV)	매 암호화마다 무작위 12 byte 생성 — nonce 재사용 없음
인증 태그	16 byte (위·변조 시 복호화 실패)
토큰 포맷	<code>base64(iv authTag ciphertext)</code>
저장 위치	ElastiCache — 평문은 메모리 외 어디에도 존재하지 않음
TTL	<code>NHIS_SESSION_TTL_SECONDS</code> (기본 600s)

4.3 증빙

- 구현: `src/common/crypto/session-cipher.service.ts` (encrypt/decrypt)
- 저장 어댑터: `src/nhis/infrastructure/persist/redis-nhis-auth-session.store.ts`
- 검증: `src/common/crypto/session-cipher.service.spec.ts` 단위 테스트 통과
- 아키텍처 문서 §5.3 「세션 데이터 암호화」

4.4 잔여 위험

- 메모리 덤프 공격: 컨테이너 메모리 덤프를 획득한 공격자는 활성 세션의 평문에 접근 가능. 컨테이너 격리(§5) 와 호스트 보안 통제로 완화.

5. 접근 통제

5.1 점검 방법

- 외부 인입 경로의 인증 통제 코드 검토 (`src/auth/guards/`)
- 네트워크 노출면 점검 (`Dockerfile` , `deploy/docker-compose.yml` , `deploy/nginx/nginx.conf`)
- 컨테이너 권한 통제 점검 (USER 지정, 불필요 바이너리 제거)

5.2 결과 — 인입 API 인증

AppCredentialsGuard 가 모든 비즈니스 endpoint 에 헤더 자격증명을 강제하고, 검증은 상수시간 비교로 수행됨.

통제	동작
대상 endpoint	POST /api/v2/nhis/start_auth · POST /api/v2/nhis/after_auth 양쪽에 @UseGuards(AppCredentialsGuard) 적용
요구 헤더	x-nhis-scrapper-access-key · x-nhis-scrapper-access-secret 두 개 모두
검증 방식	두 값을 SHA-256 해시한 뒤 crypto.timingSafeEqual 로 비교 — 상수시간 비교
불일치 시	NhisInvalidAppCredentialsError 반환 (HTTP 401)
예외	GET /health 만 가드 미적용 (LB / ALB 헬스체크용, 의존 인프라 상태만 노출)

5.3 결과 — 네트워크 노출면

외부 인입은 nginx 단일 포트로 한정되어 있으며, WAS 컨테이너는 외부 미노출.

통제	동작
외부 노출 포트	nginx 컨테이너의 \${NGINX_PORT} (기본 80) 하나만
WAS 컨테이너	외부 미노출. Docker 내부 네트워크 app 만 통과. nginx 만이 was:3000 으로 접근
Redis	본 제품 번들에 미포함 (고객사 ElastiCache). 내부망 전용 사용 권장
ElastiCache 접근	REDIS_PASSWORD / REDIS_USERNAME / REDIS_TLS 모두 환경변수로 통제

참고 — 본 제품 외부 통제 영역: ALB·보안그룹·VPC 라우팅·ElastiCache 보안그룹 등 인프라 단 통제

5.4 결과 — 컨테이너 권한

runner 이미지가 비-root 사용자로 동작하고, 불필요 바이너리·번들 패키지가 제거되어 공격 표면이 축소됨.

통제	동작
실행 사용자	USER nhis (UID/GID 분리) — 컨테이너 침해 시 호스트 권한 상승 차단
npm/npx 제거	런타임 이미지에 npm/npx/yarn 미포함 (/usr/local/lib/node_modules/npm , /usr/local/bin/{npm,npx} , /opt/yarn-v* 모두 삭제). 부산 효과로 ip-address@10.1.0 등 npm 번들 패키지의 잠재 취약점도 제거
최소 베이스 이미지	node:22-alpine (multi-stage build). 빌드 종속성은 builder 단계에서만 사용되고 runner 이미지에 미포함

5.5 증빙

- 가드 구현: `src/auth/guards/app-credentials.guard.ts:13-33`
- 검증: `src/auth/guards/app-credentials.guard.spec.ts`
- Dockerfile (USER, npm 제거): `Dockerfile` (runner stage)
- nginx 외부 노출 설정: `deploy/docker-compose.yml` , `deploy/nginx/nginx.conf`
- 아키텍처 문서 §5.2 「외부 API 접근통제」 · §5.5 「컨테이너 보안」

5.6 잔여 위험

- IP allow-list 미사용 — 헤더 자격증명만으로 통제. 자격증명이 유출되면 외부 어디서나 호출 가능. 인입 경로 IP 제한(보안그룹·WAF) 을 인프라 단에서 함께 적용 권장.
- 자격증명 회전 정책은 본 제품이 강제하지 않음 — 운영 정책으로 관리.

6. NHIS · 외부 통신 보안 (부가)

6.1 결과

NHIS 호출은 HTTPS 전용이며, cookie jar 는 세션별로 격리되어 있음.

통제	동작
TLS	NHIS 엔드포인트(<code>www.nhis.or.kr</code> , <code>m.nhis.or.kr</code>) 가 HTTPS 이므로 모든 호출은 HTTPS 로 수행됨. HTTP/HTTPS agent 는 대상 URL 스킴에 맞춰 자동 선택
Cookie 격리	세션별 <code>tough-cookie</code> CookieJar — 인스턴스 간·세션 간 cookie 누수 없음
프록시 (선택)	<code>NET_PROXY_URL</code> 지정 시 단일 출구 프록시 경유 가능 — 격리망에서 출구 IP 통제
타임아웃	<code>NET_REQUEST_TIMEOUT_MS</code> (기본 30 초) 로 무한 대기 방지
응답 본문 로깅	미기록 (개인정보가 포함되므로 메모리 외 영구화 금지)

6.2 증빙

- 구현: `src/nhis/infrastructure/http/nhis-http-session.ts`
- 아키텍처 문서 §5.6 「NHIS 통신 보안」

7. 종합 결론

본 제품의 4대 점검 항목 결과 요약:

항목	결과
취약점 점검 (OSS)	Critical / High / Medium / Low 합계 0건 (Docker Scout + Gype 스캔, DB v6.1.4 / 2026-05-27 기준)
로그 마스킹	allowlist (LOG_SAFE_KEYS) + 그 외 leaf 값 maskValue 자동 적용 (첫·끝 글자 제외 * 치환). 응답 본문 로깅 없음
데이터 암호화	세션은 AES-256-GCM (key 256bit, IV 12byte 무작위, authTag 16byte) 으로 ElastiCache 저장. 평문은 메모리 외 미존재
접근 통제	비즈니스 endpoint 전체 AppCredentialsGuard 적용, SHA-256 해시 + timingSafeEqual 상수시간 비교. 외부 노출은 nginx 단일 포트, WAS 컨테이너 미노출. runner 이미지 비-root + npm/npx/yarn 제거

참고 — 본 제품 외부의 보안 통제 항목:

- 호스트 방화벽·보안그룹에서 nginx 노출 포트 외 모든 인입 차단
- ElastiCache 보안그룹은 WAS 호스트만 허용. private subnet 권장
- 외부 → 인입 트래픽의 TLS 종단 위치 결정 (ALB · nginx · 또는 미적용)
- 권장 토폴로지: ALB internal / WAS private subnet / NHIS 발신은 NAT Gateway 경유. 실제 배치는 고객사 환경에 따름